FORTH
INSTITUTE OF COMPUTER SCIENCE

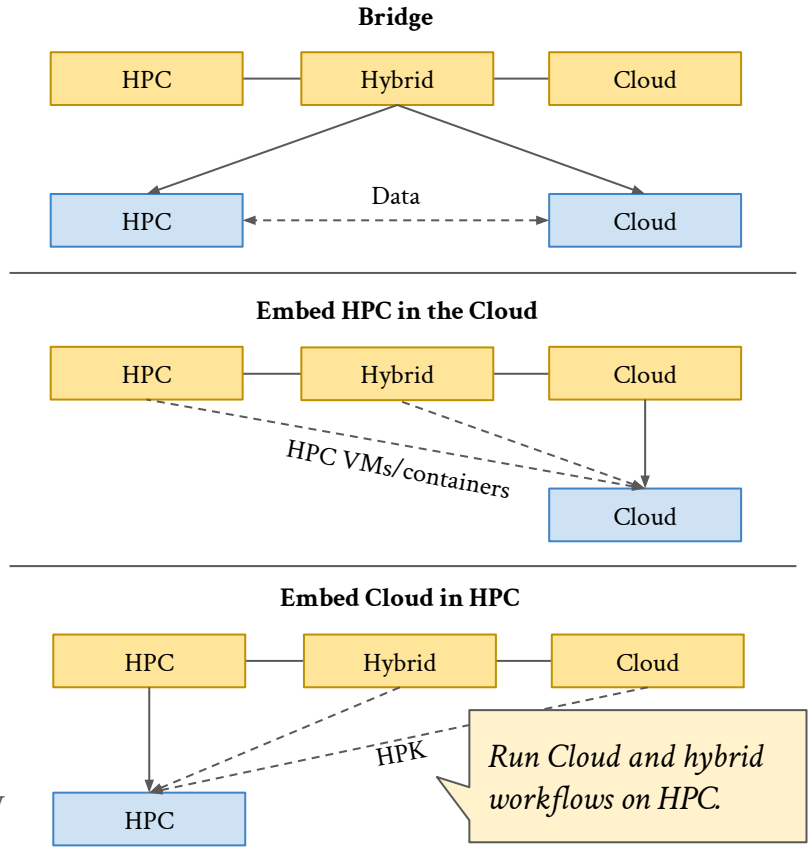
Evaluating HPK for Running Cloud-Native Workloads on Slurm Clusters

17 November 2025 • CANOPIE-HPC 2025 • SC25

Antony Chazapis, L. Vassilakis, G. Petsis, M. Marazakis, A. Bilas
FORTH-ICS, Greece

Running hybrid workloads

- Bridge the two separate environments
  - Submit HPC jobs from the Cloud side or vice versa[1]
  - Deal with separate data and network contexts, unless running HPC in the Cloud (VM offerings available)
  - Hardware and maintenance costs
- Run both on the same hardware
  - Synchronize schedulers or use either for scheduling
- Run Slurm in Kubernetes
  - Embed the HPC software stack in containers[2]
- Run Kubernetes in Slurm with HPK!
  - Create ephemeral Kubernetes mini-Clouds in HPC[3]
  - Why? → EuroHPC, stability/consistency, multi-tenancy



[1] KNoC is a Kubernetes node to manage container lifecycle on HPC clusters: https://github.com/CARV-ICS-FORTH/knoc (InteractiveHPC 2022)
[2] Genisys is a Kubernetes scheduler for running HPC jobs inside Virtual Clusters alongside other services: https://github.com/CARV-ICS-FORTH/genisys (VHPC'22)
[3] HPK allows users to run Kubernetes in Slurm: https://github.com/CARV-ICS-FORTH/HPK (WOCC'23)

Design goals

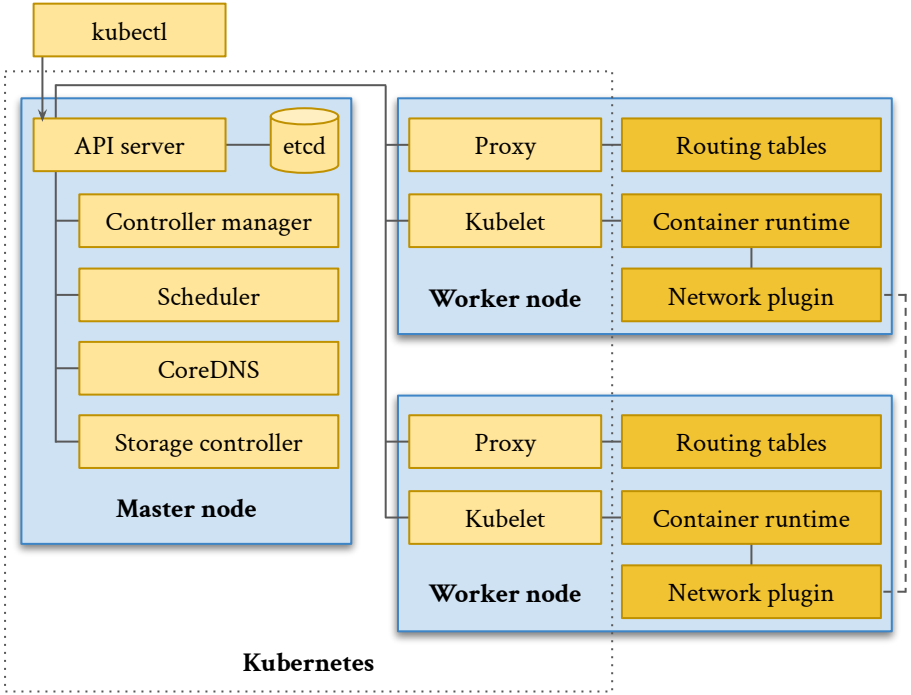
HPK → Run *Kubernetes in an HPC environment* as a user

Requirements:

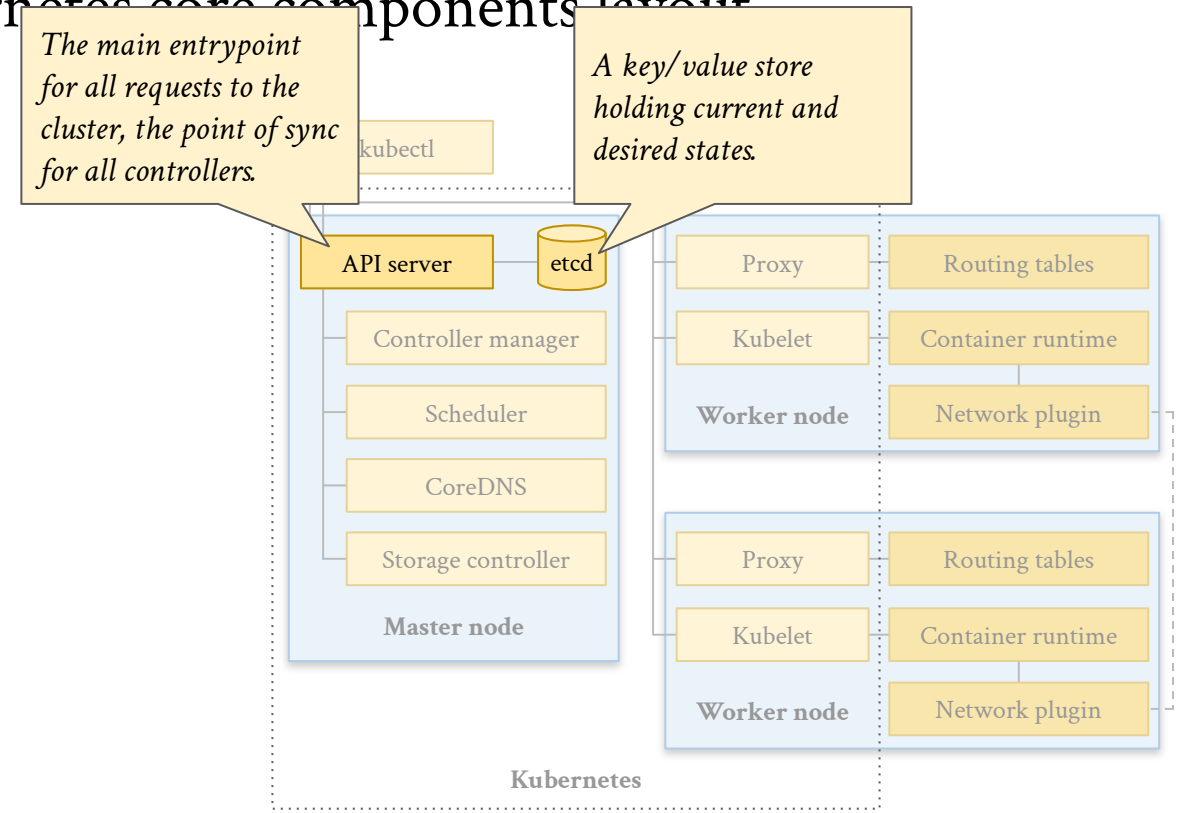
- All* Kubernetes abstractions should be available and fully functional
  - *Except those that relate to physical hardware resources (like NodePort services)
  - Private, inter-container network and internal DNS should work as expected
- Delegate resource management to Slurm
  - Respect organization policies
  - Comply with established resource accounting
  - Scale across all nodes of the cluster
- Use Singularity as the container runtime
  - Preinstalled in most HPC environments
- Make it easy for HPC administrators
  - No (or little) configuration changes should be required at the host level
  - No reliance on special libraries or binaries that execute with "elevated" permissions
- Make it easy for users
  - All neatly packaged up in one container
  - Simple, one-command deployment via Slurm
  - All relevant configuration and files in $HOME

Kubernetes technical concepts

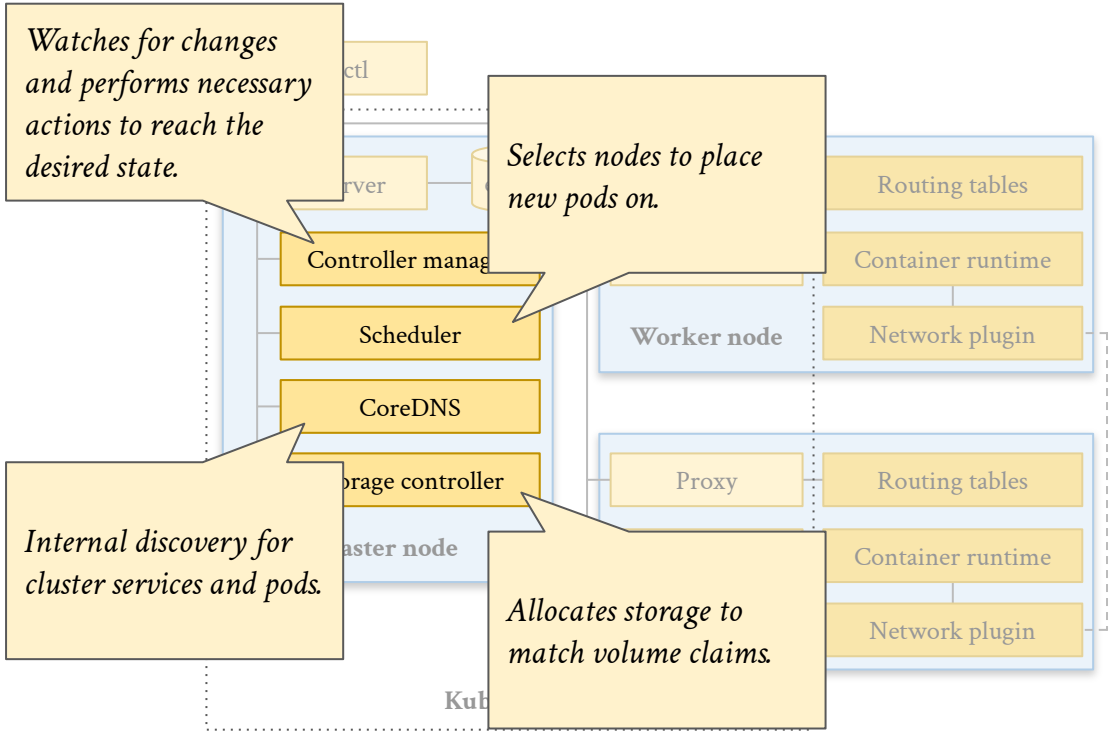
- Single API endpoint
- Multiple controllers/operators
- Objects stored in etcd
  - Pods → Collection of containers
  - Deployments → Replicated pod groups
  - Services → Microservice naming
  - Jobs → Pods that run to completion
  - Volumes → Mountable file collections
- Typical distributed structure
  - "Control plane" → Scheduling and placement
  - Node agents → Handle execution



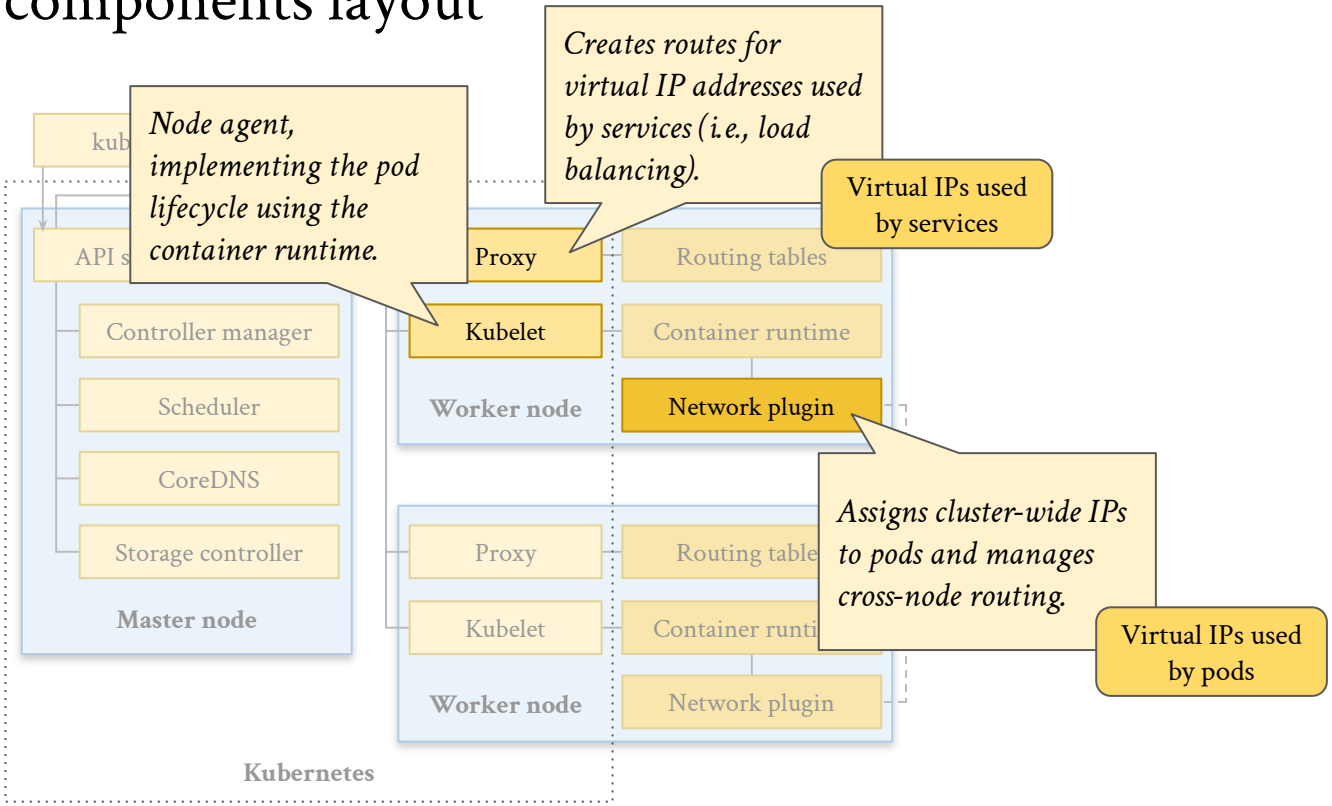
Kubernetes core components layout



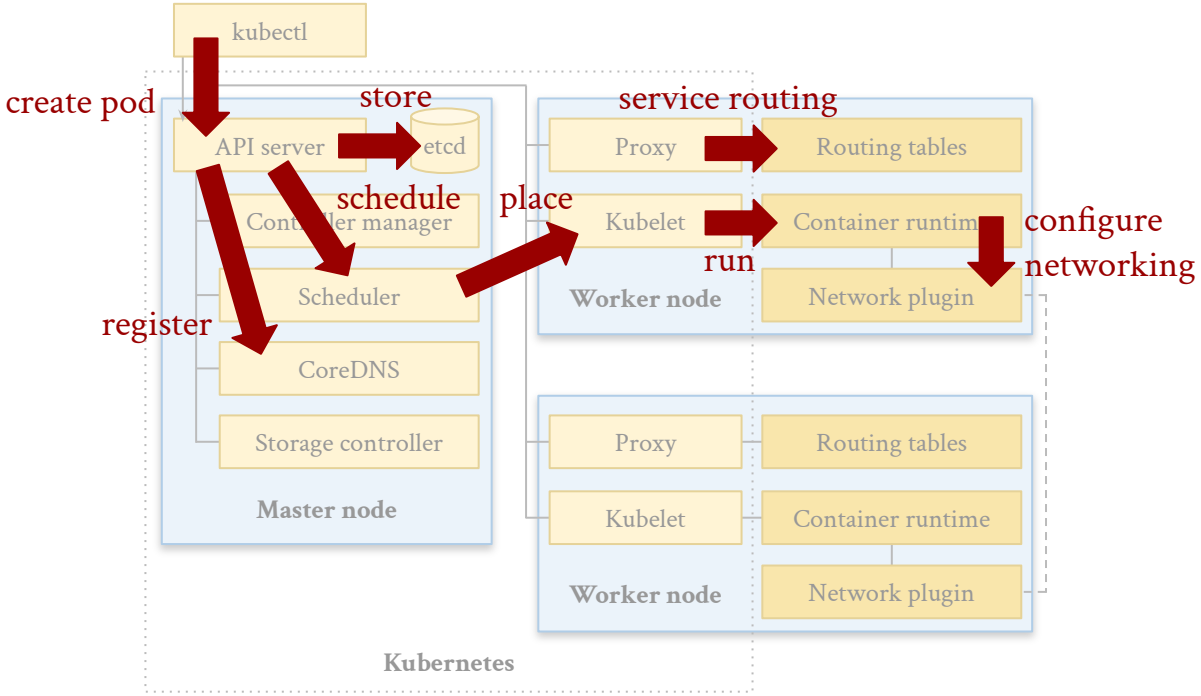
Kubernetes core components layout



Kubernetes core components layout

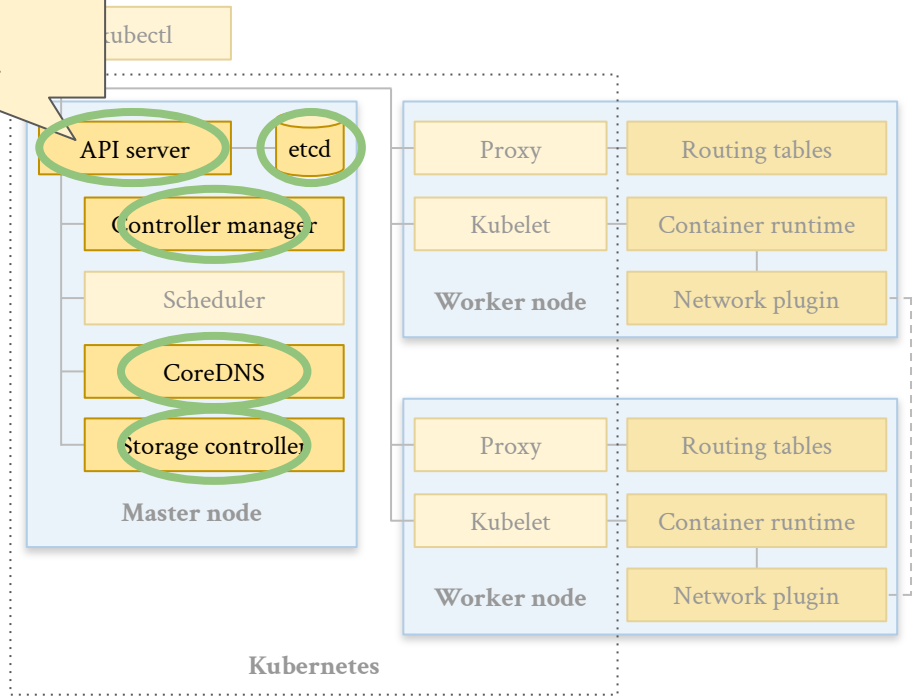


Kubernetes core components layout

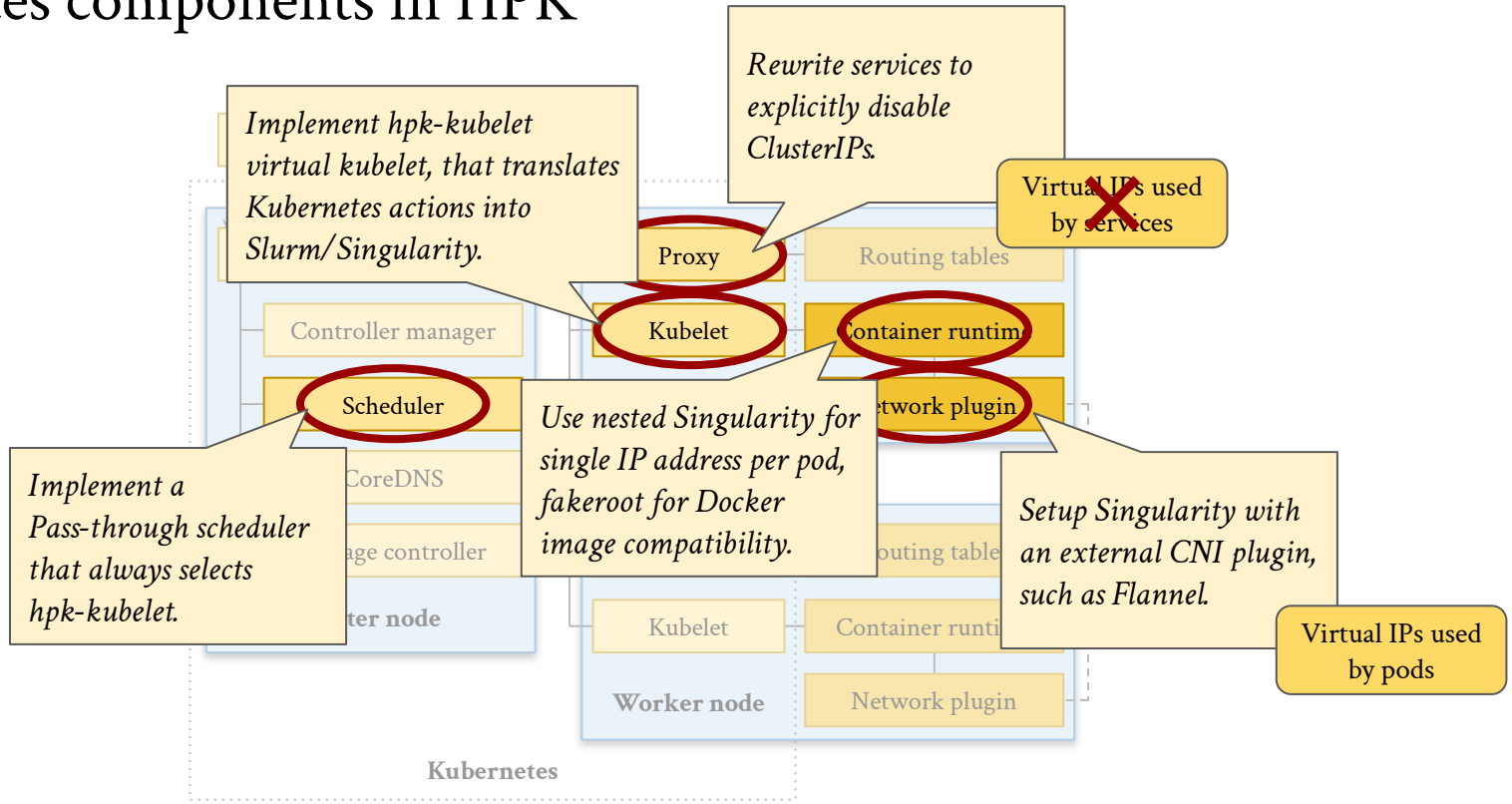


Kubernetes components in HPK

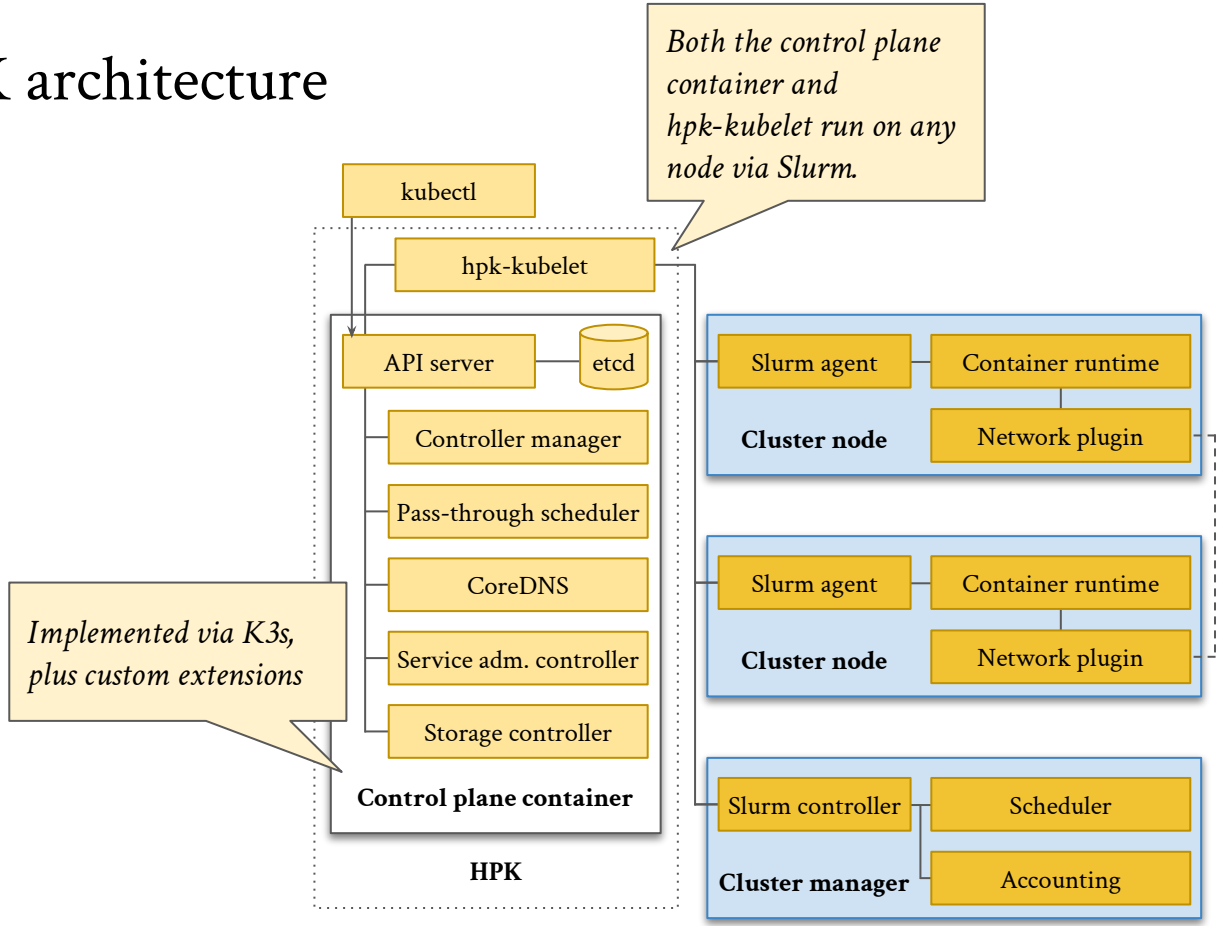
Services that run "out of the box", with no special dependencies, or minimal changes.



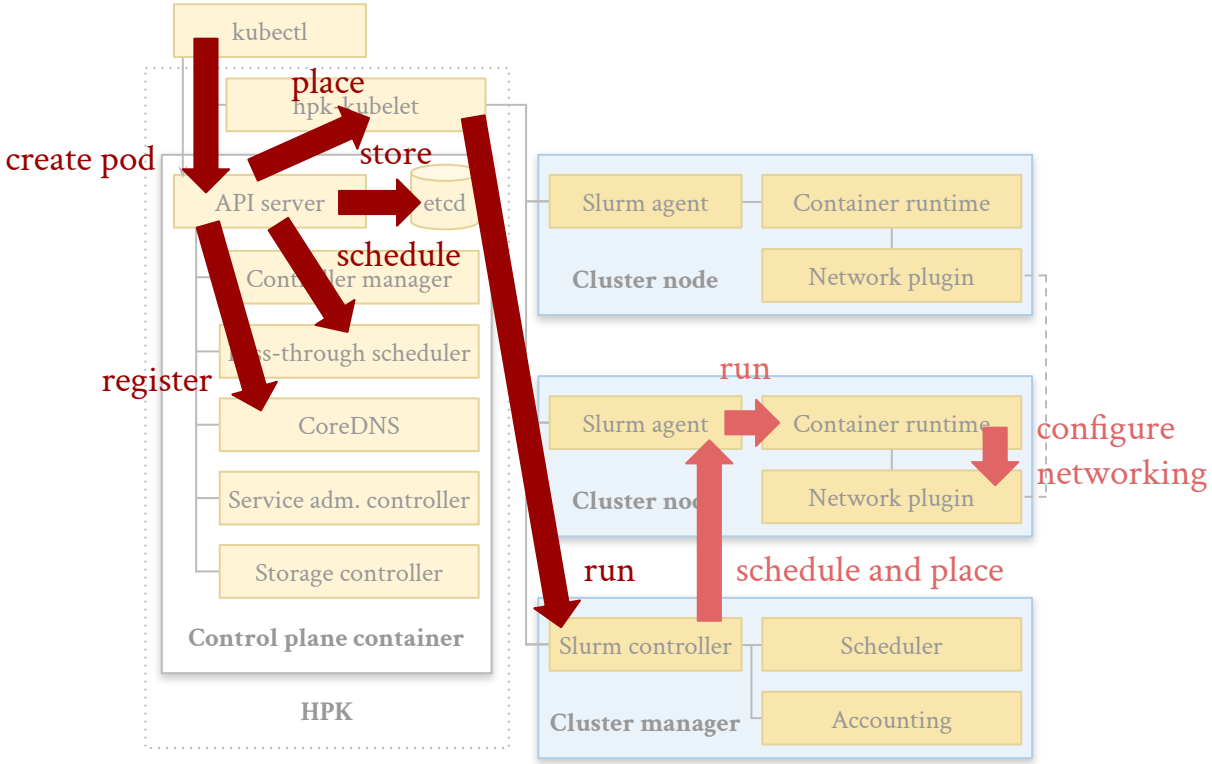
Kubernetes components in HPK



HPK architecture

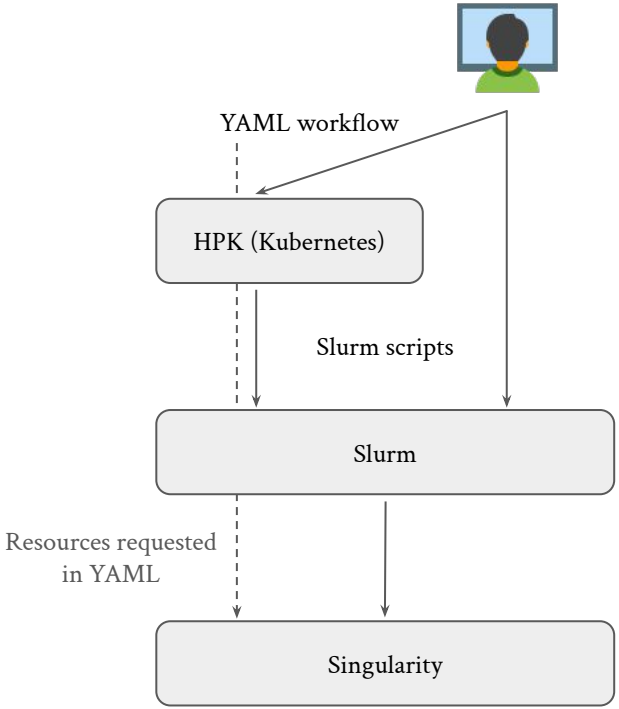


HPK architecture



HPK implementation

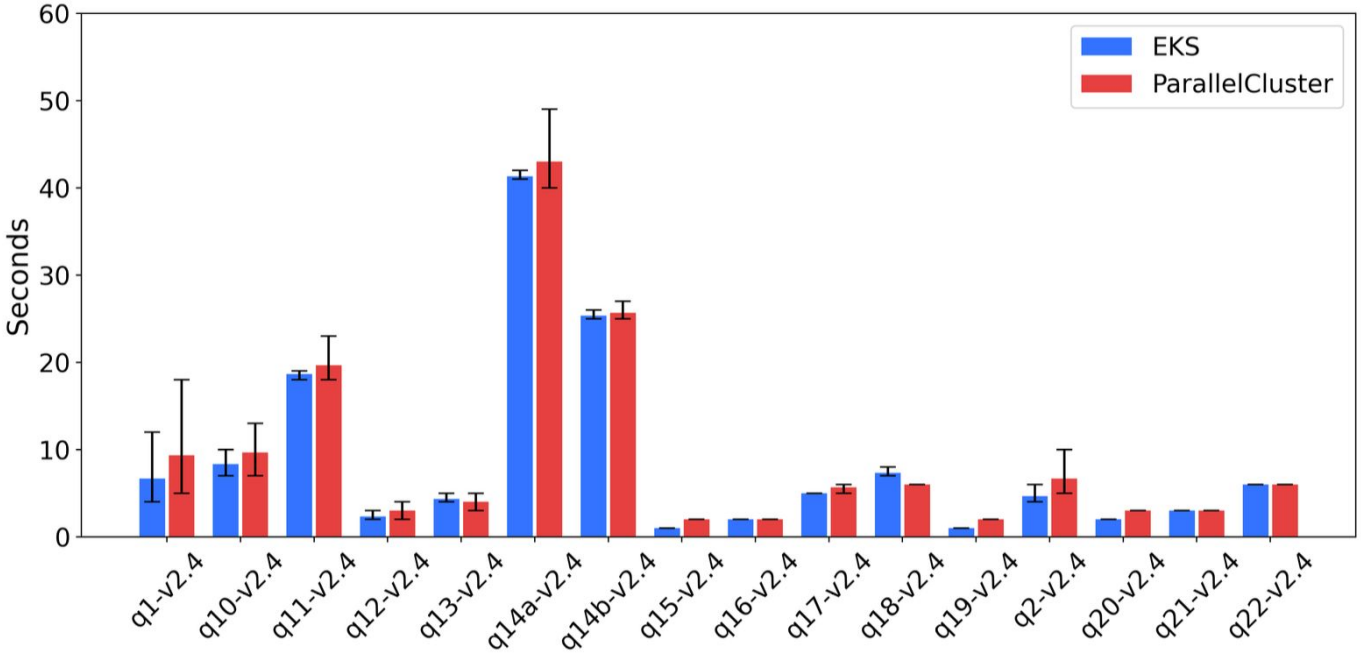
- HPK translates Kubernetes to Slurm scripts
  - Pods/jobs enter as YAML through the Kubernetes API, exit as Slurm scripts from hpk-kubelet → Pods show up as Slurm jobs
  - Slurm jobs run a hierarchy of Singularity containers (containers within pods) → Pause container is the pod
  - Kubernetes resource requirements end up in Slurm allocation requests → No changes to accounting
- HPK runs as a user process via Slurm
  - User can run both Kubernetes and Slurm workloads at the same time
  - No "special" allocation needed for HPK → 1 CPU, few GBs of RAM should be enough
  - Little support needed by the environment → No Slurm modifications, some Singularity networking configuration (inc. Flannel or other CNI)



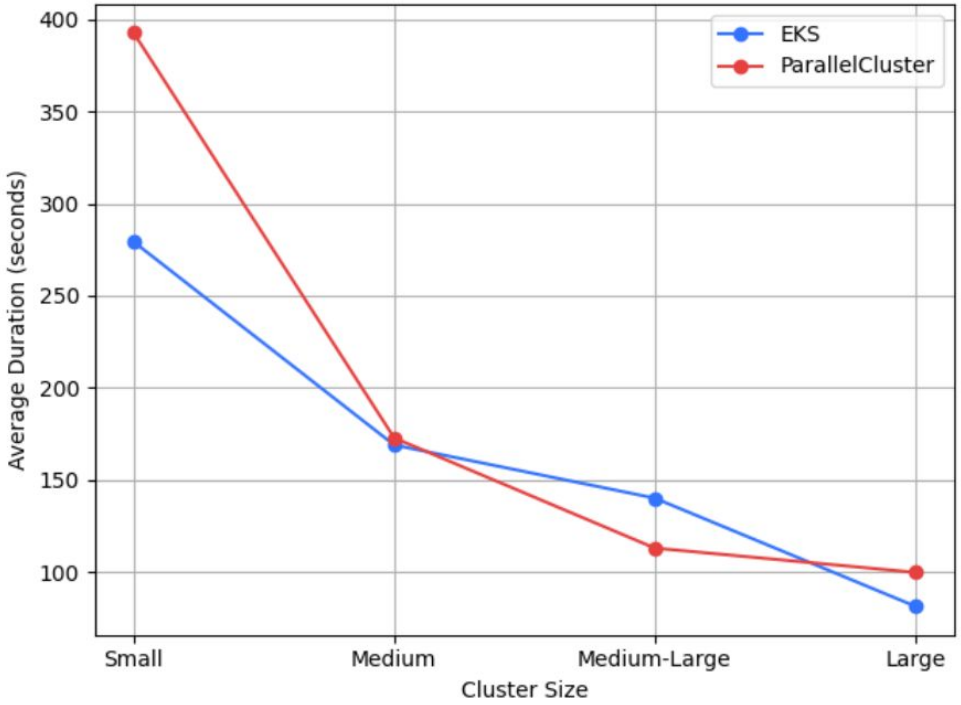
Evaluation

- Same applications on same hardware
  - EKS vs ParallelCluster on AWS
  - Applications deployed via Helm
  - Other software utilities may be necessary → MinIO
- Spark TCP-DS
  - Spark Operator → SparkApplication CRD
- Dask matrix multiplication
  - Varying configurations of driver/workers
- PyTorch model fine-tuning
  - Runs in a Jupyter notebook
  - Kubeflow Training Operator → PyTorchJob CRD
  - CPU-only vs GPU execution

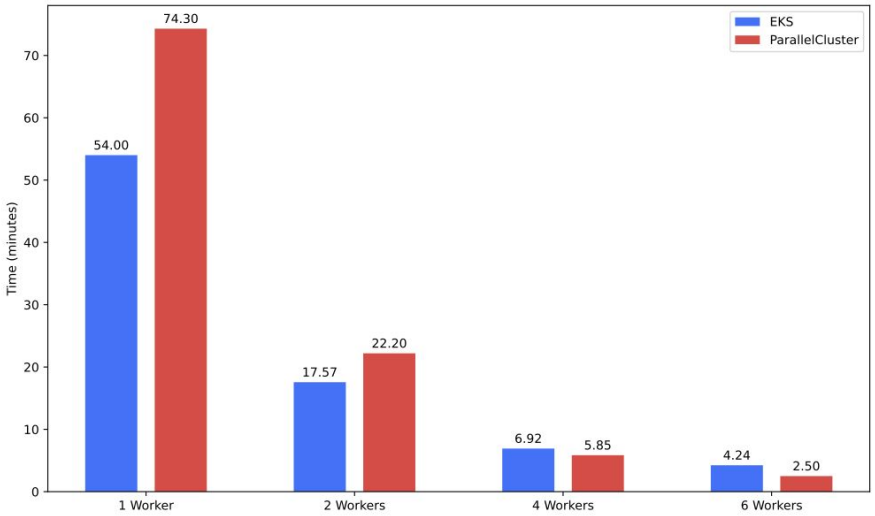
Spark TCP-DS


EKS
ParallelCluster
Seconds
0
10
20
30
40
50
60
q1-v2.4
q10-v2.4
q11-v2.4
q12-v2.4
q13-v2.4
q14a-v2.4
q14b-v2.4
q15-v2.4
q16-v2.4
q17-v2.4
q18-v2.4
q19-v2.4
q2-v2.4
q20-v2.4
q21-v2.4
q22-v2.4

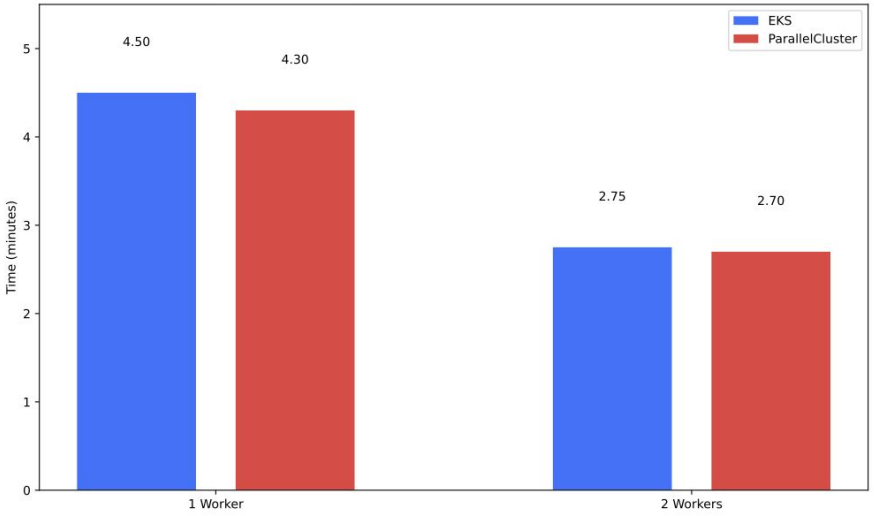
Dask


EKS
ParallelCluster
Average Duration (seconds)
400
350
300
250
200
150
100
Small
Medium
Medium-Large
Large
Cluster Size

PyTorch



CPU



GPU

Overview

- Running Kubernetes in Slurm is possible!
  - Currently working on integrating with large EuroHPC systems
  - Exploring options to avoid all system configuration requirements
  - Planning to contribute the project to HPSF
- HPK successfully deploys a variety of applications/utilities (also via Helm)
- Performance is similar to native Kubernetes (often even better!)
- Vision
  - Cloud-user PoV → Run on HPC hardware
  - HPC-user PoV → Exploit the Cloud software ecosystem
  - HPC centre PoV → Run Cloud workloads on the main HPC partition, attract users

Try it out! Feel free to contact us!

https://github.com/CARV-ICS-FORTH/HPK

Acknowledgements

DaFAB Riser grnet